MODULE SPECIFICATION & PARTICIPANT GUIDE

Module 1: RAG Chatbot Builder

The Engineer's Code 2026 — Directorate of AI, NED University

TEAM COMPOSITION	DEVELOPMENT WINDOW	SUBMISSION MEDIUM	EVALUATION FORMAT
2 to 3 registered members	4 continuous hours (On-site)	Source code via email (Day 1)	5–10 min presentation & live demo

Primary Objective: Teams receive an authoritative source document in PDF format at the start of the session. The requirement is to engineer an accurate, polite, and fully grounded Retrieval-Augmented Generation (RAG) system capable of synthesizing factual responses strictly from the provided text.

UNIVERSAL EVENT REGULATIONS

STANDARD OPERATING PRINCIPLES

- Participation is restricted entirely to registered team members. Collaboration with external individuals or remote contacts is strictly prohibited.
- All development must take place within the venue during the official allocated block; pre-prepared modules or post-deadline submissions will not be accepted.
- Participants must supply their own hardware, network setups, and required API credentials. Standard AI developer tooling (Claude, OpenAI, Gemini, Cursor) is permitted.
- Teams are accountable for technical defenses. Evaluations are oral; judges assess architectural choices, chunking trade-offs, and live functionality rather than passive slide decks.

MODULE REGULATIONS & TECHNICAL BOUNDARIES

PERMITTED TECHNOLOGIES & WORKFLOWS

- Open-source or proprietary orchestration frameworks (LangChain, LlamaIndex, or native Python implementations).
- Any vector database or indexing solution (FAISS, ChromaDB, LanceDB, Qdrant).
- Standard embedding models and foundational LLM endpoints.
- Interactive interfaces suited for live defense (Streamlit, Gradio, or standard CLI/terminal interfaces).

STRICTLY PROHIBITED PRACTICES

- Hallucination or relying on baseline parametric memory rather than retrieved source context.
- Hardcoding deterministic responses or scripting predictable test-case mappings.
- Bypassing vector retrieval by injecting the entire document verbatim into each prompt window.
- Unauthorized sharing of datasets, repositories, or source code across competing teams.

System Requirements & Deliverables: The final implementation must enforce strict grounding, preserve professional tone under prompt manipulation, and explicitly refuse out-of-scope inquiries. Submissions must include repository code, reproducible dependency manifests (`requirements.txt`), and operational run instructions.

RECOMMENDED TECHNICAL PIPELINE

01 **Document Ingestion:** Thoroughly review the target PDF to identify formatting eccentricities, tables, headings, and domain scope.

02 **Text Extraction:** Parse structured text cleanly using reliable parsing libraries (e.g., `pdfplumber`, `PyPDF2`, or `pymupdf`).

03 **Chunking Architecture:** Segment parsed text into optimal chunk boundaries. Determine chunk size and overlap ratios, preparing defensible rationales for chosen thresholds.

04 **Vector Embedding & Indexing:** Generate vector embeddings and store index representations within your chosen vector database.

05 **Retrieval Verification:** Validate semantic similarity searches in isolation. Ensure target queries return contextually complete and high-precision chunks.

06 **System Prompt Design:** Construct strict system guardrails enforcing document-only grounding, clear refusal when data is absent, and prompt-injection resistance.

07 **Inference & Interface Integration:** Couple the retrieval chain with the LLM API and establish a functional, responsive interface for real-time interaction.

08 **Robustness & Adversarial Testing:** Stress test the pipeline against edge cases, hostile prompts, out-of-domain inquiries, and ambiguous references.

09 **Packaging & Dispatch:** Archive clean source code, run instructions, and dependency lists, transmitting the submission prior to the deadline cutoff.

10 **Defense Preparation:** Prepare concise justifications detailing framework selection, chunking parameters, latency trade-offs, and failure recovery mechanisms.

PARTICIPANT GUIDANCE & FAQ

Q: Are generative assistants permitted during code implementation?
Yes. AI assistants (ChatGPT, Claude, Cursor, Antigravity) are fully allowed, provided all team members understand every architectural choice and can defend the underlying code during oral examination.

Q: Is graphical UI quality weighted heavily during assessment?
No. Working functionality, retrieval accuracy, and system grounding are the primary criteria. A clean terminal CLI or basic Streamlit interface is entirely acceptable.

Q: How should teams address complex visual assets or non-tabular images?
Prioritize clean textual chunking first. If specific non-textual components remain unindexed, acknowledge these architectural boundaries transparently during evaluation.

Q: What protocol applies when information requested is not present in the document?
The system must refuse clearly and state document omission. Unsubstantiated model confabulation (hallucination) is evaluated significantly more harshly than an accurate refusal.

Q: Are evaluation panels authorized to issue custom or unseen prompts?
Yes. Evaluators will cross-examine systems with unexpected, out-of-scope, and adversarial queries during the live defense.

Q: What contingency exists if an API quota exhausts during the competition window?
Notify the designated module volunteer immediately and migrate to an alternative backup key or pre-configured secondary model provider.